Evolutionary Graph Fusion Architecture Search

1st Aohan Mei
School of Software
Northeastern University
Shenyang, China
2471436@stu.neu.edu.cn

2nd Nan Li
School of Software
Northeastern University
Shenyang, China
2010500@stu.neu.edu.cn

3nd Tian Zhang
School of Software
Northeastern University
Shenyang, China
2190087@stu.neu.edu.cn

4th Lianbo Ma*
School of Software
Northeastern University
Shenyang, China
malb@swc.neu.edu.cn

2025 IEEE Congress on Evolutionary Computation (CEC) | 979-8-3315-3431-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/CEC65147.2025.11043065

***Abstract*—The great success of graph neural networks (GNNs) in graph-structured data tasks benefits from the powerful structure learning abilities of their architectures. For complex datasets, too deep GNNs can suffer from over-smoothing problem, leading to degradation of prediction performance. Recently, a graph fusion architecture designed by domain experts mitigates the over-smoothing problem. However, this design paradigm is labor and computation-intensive. Drawing on the idea of neural architecture search, this paper proposes evolutionary graph fusion architecture search (EGFAS), which can automatically find graph fusion architecture with the capability to solve the over-smoothing problem. Specifically, we design a graph fusion architecture search space, which includes multiple aggregation functions and fusion operations. An efficient encoding method is used to represent candidate architectures. Furthermore, we propose a surrogate model to solve the expensive overhead of performance evaluation. Experiments on five benchmark datasets are carried out to confirm the superiority of the proposed algorithm. It is shown that the proposed method leads to better results comparing to the state-of-the-art algorithms in terms of search effectiveness and adaptability to different datasets.**

***Index Terms*—Graph Neural Network, Neural Architecture Search, Evolutionary Computation, Performance Predictor.**

I. Introduction

Graph neural networks (GNNs) have achieved success in various graph-structured data tasks, including social network analysis [1], traffic flow forecasting [2], and protein prediction [3]. As a deep learning model capable of effectively capturing the features of graph-structured data and the relationships between nodes, GNN roughly follows the scheme of propagating and aggregating messages between nodes and then updating the nodes. In order to capture more structural features of graphs [4], GNNs attempt to perform multiple aggregation operations [5]. However, as the number of aggregation operations increases, excessive message propagation can lead to the convergence of node representations, especially when the graph has a regular structure or strong connectivity between nodes. This causes the nodes to lose their original features and discriminability, known as the over-smoothing problem in GNNs [4]. To mitigate this problem, many works design skip connections, allowing nodes to selectively receive global information without losing discriminability. Examples of such works include ResGCN [6], JK-Net [7], and DAGNN [8]. However, such manual design of skip-connection usually relies on substantial human expertise [9]. Moreover, given the diversity of possible topologies and the other modules such as aggregation, manually designing GNNs actually has some drawbacks.

To address this challenge, we utilize the capabilities of neural architecture search (NAS), which has proven effective in designing data-specific CNNs [10], [11], [12] to achieve the adaptive topology design. In fact, there have indeed been some works that have attempted to use NAS to automatically design GNN model architectures recently [13], [14], [15]. However, in addition to the basic topologies representing the transmission paths of message, existing GNN-oriented NAS methods focus primarily on shallow searches such as aggregation, combination, and activation, while these methods have been proven effective, they often process features in isolation and lack a mechanism to comprehensively integrate information from different levels of the graph. That is, the topology and operation are separated into two separate individual searches. Although this does help control the flow of messages within a GNN to some extent, it has certain limitations on the capacity of the model. Many NAS works increase the search space complexity to address this issue, typical candidate operations include feature transform functions [14], global attention pool, global attention sum pool [16], and so on, but the search costs will also increase accordingly.

Instead of simply increasing the search space complexity, we hope to solve the lack of a comprehensive integration mechanism for different levels of information across graphs. In our approach, a new perspective on feature fusion serves as a key mechanism to address this gap. Specifically, this fusion process enables the selective combination of features from multiple levels of the graph, enriching the representation of each node. We know that common GNNs include GCN [5], GraphSAGE [17], GAT [18], and GIN [19]. Their aggregation methods vary, but all rely on basic operators such as sum, mean, and max to aggregate, combine, and update the features of nodes based on messages from neighboring nodes [20]. Since this approach has been proven effective, similarly, we can also "aggregate" adjacent nodes in the GNN topology, fusing features from different levels that have been pre-selected through predefined operations. By doing so, we can take full advantage of the diversity of GNN topologies and make the message propagation more selective within a

* indicates corresponding author.

979-8-3315-3431-8/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on May 07,2026 at 16:16:02 UTC from IEEE Xplore. Restrictions apply.

lightweight structures to save search costs, ensuring that it is both effective and computationally efficient.

Common methods for NAS can be divided into three main categories: 1) gradient-based methods (denoted as GDNAS) [21], [22]; 2) reinforcement learning (RL)-based methods (denoted as RLNAS) [11], [23]; and 3) evolutionary algorithm (EA [24], [25])-based methods (denoted as ENAS) [26], [27], [28]. Although GDNAS can sometimes be faster [29], it often requires expensive computational memory, which results in various restrictions on the search space [30]. In contrast, RLNAS and ENAS require extensive evaluation, making them less efficient, but they possess powerful black-box optimization techniques [31]. Among them, ENAS can effectively avoid local optimal solutions and is suitable for complex and discrete search spaces [32].

Thus, this work proposes a novel evolutionary graph fusion architecture search (EGFAS) by combining topology, feature fusion strategy, and aggregation operation of GNN. After the topology is designed to determine the path of message propagation, the framework further processes the messages at different levels that converged to one point through fusion operations and finally aggregates the resulting final information to adaptively design data-specific GNNs that are both light and capacitive. Then, the challenge becomes reducing the high computational cost of evaluating multiple candidate networks during the evolution process. To address it, we propose a new encoding scheme. In ENAS, each individual corresponds to an architecture, and architectures are encoded as a set of hyperparameters. Common approaches include graph-based encoding and path-based encoding. We design a novel graph-based encoding scheme, expanding from the perspective of nodes in GNNs. For each node with topological relationships in a GNN, we directly connect its corresponding topological information with fusion and aggregation encodings, effectively representing the complex information contained within a neural network.

Additionally, we use a cheap offline classifier as a surrogate model to assist the evolution process, aiming to replace the time-consuming real training with its ability to predict the performance of candidate architectures. We also provide a method for constructing its training set and a score-based fitness evaluation scheme.

In summary, the main contributions of this paper are summarized as follows.

1) A novel EGFAS is proposed to automate the construction of the graph fusion component. In addition, this paper designs new encoding method and fitness evaluation method to enhance the efficiency of EGFAS.
2) We conduct extensive experiments of EGFAS on five datasets with various state-of-the-art algorithms. The experimental results show that the architecture designed by EGFAS has significant improvements on accuracy.

II. Backgrounds

A. Graph Neural Architecture Search (GNAS)

As an automated method, NAS optimizes the design of neural network architectures through search algorithms, aiming to automatically discover architectures that outperform manually designed ones. It has shown promising results in searching for CNNs [10], [11], [12]. In recent years, to obtain data-specific GNN architectures, several GNAS approaches have been proposed. Categorized from search strategy, GraphNAS [14] and Auto-GNN [33] were the first to introduce NAS into GNNs, employing RL methods. The EA based GNAS leverages the global search capability of evolutionary algorithms [32], [34], effectively avoiding local optima. Considering search efficiency, gradient-based differentiable architecture search (DARTS) [13], [22] has also been applied to optimize GNNs. By introducing a continuous search space, it utilizes gradient descent to efficiently search for optimal architectures, significantly reducing search time and computational cost. In recent progresses, Yao et al. [35] design data-augmented curriculum GNAS for learning an architecture with good generalizability under distribution shifts. Gao et al. [36] utilizes the learning power of LLM to quickly evaluate candidate architectures. Wang et al. [37] automates the design of GNNs for out-of-distribution generalization.

B. Aggregation and Fusion in GNN

Aggregation operation. In GNNs, the aggregation function is a core operation for information propagation in graph-structured data, used to aggregate features from neighboring nodes. These functions need to satisfy permutation invariance (i.e. insensitivity to the order of neighboring nodes) and locality (only depending on information from neighboring nodes). Common aggregators include the sum aggregator, mean aggregator, max aggregator, weighted sum aggregator, LSTM aggregator, pooling aggregator, and MLP aggregator. Based on these aggregators, a series of implementation paradigms for information propagation and aggregation mechanisms have been designed, which can be easily integrated into modern deep learning frameworks such as PyTorch and TensorFlow. For example, GAT [18] employs weighted sum aggregation, where weights are dynamically computed via an attention mechanism; the classical GCN [5] uses weighted mean aggregation, achieved by normalizing the adjacency matrix; GIN [19] emphasizes the combination of neighbor and self-information, using sum as the basic aggregation function and applying an MLP afterward to learn non-linear relationships.

Fusion operation. A single aggregation function may not fully capture the diversity of neighborhoods in a graph. To enhance the expressive power of the model, fusing the outputs of multiple intermediate aggregation layers is proposed. In existing GNN studies, fusion operations are primarily focused on summation and concatenation. Representative models using concatenation include PNA [38], MixHop [39], and JK-Net [7], while those using summation include GCNII [40] and ResGCN [6].

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on May 07,2026 at 16:16:02 UTC from IEEE Xplore. Restrictions apply.

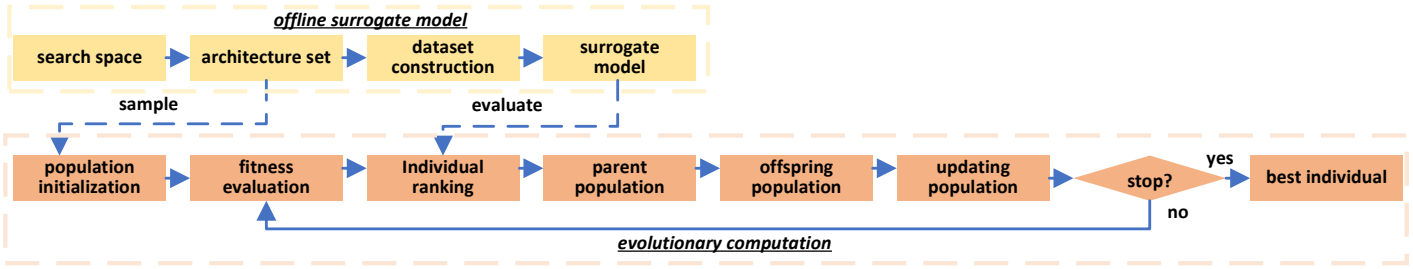


Fig. 1. The overall framework of EGFAS.

C. Encoding

In ENAS, the encoding method for neural networks is used to represent network structures for search algorithms to operate on. Common encoding schemes include adjacency matrix-based encoding, [41], [42], [43], [44] and path-based encoding [45], [46], [47]. In the adjacency matrix-based scheme, the structure of a neural network is encoded as a directed acyclic graph (DAG), where nodes represent operations and edges represent connections. The encoding is created by flattening the adjacency matrix and concatenating it with a list of operation labels for the nodes. The encoding method of the important benchmark dataset NAS-Bench-101 [41] in NAS research field is completely graph-based. This method clearly expresses complex network topologies but is more computationally complex. In contrast, the path-based encoding scheme represents the neural network architecture as a string, arranging operations (e.g., aggregation, activation) and connections (e.g., skip connections) sequentially from input to output nodes. This method is simpler but can only model topology information implicitly.

D. ENAS via Performance Predictor

ENAS benefits from global search capabilities, effectively avoiding local optima. However, this advantage comes at the cost of high computational expenses due to the large number of candidate architectures generated during the evolutionary process. For example, on the CIFAR10 benchmark dataset [48], large-scale evolution algorithm [49] consumed 250 GPUs, while the hierarchical evolution algorithm [50] used 200 GPUs. Therefore, researchers suggest using surrogate models to alleviate this drawback. This approach aims to use inexpensive approximate regression or classification models to predict fitness values instead of time-consuming real training. Surrogate-assisted evolutionary algorithms are divided into two types: online and offline. The online algorithms [51], [52], [53] continuously sample new training data during the evolutionary process, gradually improving the generalization ability of the surrogate model, while the offline algorithms [54], [55], [51] separate the search process entirely from the process of fitting the surrogate model. Additionally, approximate regression models [54], [10], [51] generally predict the absolute architecture performance accurately, whereas classification models determine which architecture performs better when faced with two [55], [56], [57] or more [52] candidate models.

III. Our Approach

A. Framework

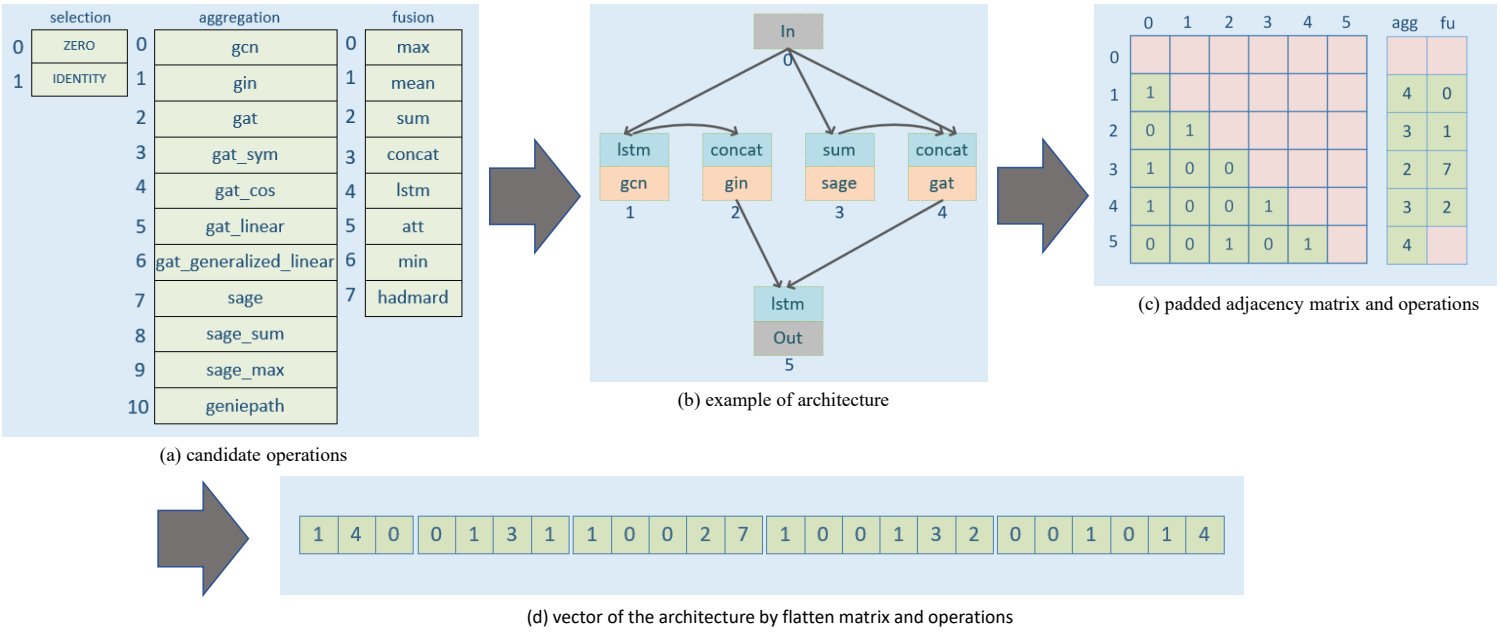
The main framework of EGFAS is outlined in Fig. 1. Specifically, to explore GNNs with superior performance, we design a search space (line 1) and sample an initial set of architectures from the proposed search space (line 2). After evaluating these architectures (line 3), a training set for the performance predictor is constructed (line 4), which the predictor is subsequently trained (line 5). Then, we randomly sample an initial population from the initial set of architectures (line 6). Following this, our algorithm enters the evolutionary search process (lines 8-15). In each iteration, a test set is first constructed from the population (line 9), and then the predictor ranks the individuals in the test set based on their fitness values (line 10). Parents are selected based on this ranking (line 11), and offspring are generated through genetic operations (crossover and mutation) (line 12). Then the population updates (line 13) before proceeding to the next iteration. When the termination condition is met, the final population is used to select the best individual, which are decoded to generate the final GNN (line 16). The pseudocode is shown in Algorithm 1.

B. Search Space and Encoding

Search space. EGFAS addresses the lack of comprehensive integration of information from different levels of the graph by introducing a feature fusion mechanism and combining it with topology. Meanwhile, it designs aggregation methods for each node to construct a GNN. Accordingly, our search space is divided into three parts: selection operations (representing topological relationships), aggregation operations, and feature fusion operations, which can be described as follows,

- Selection operations: For each node in the DAG, there are only two possibilities regarding whether it accepts its predecessor nodes as input: either select or not select.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on May 07,2026 at 16:16:02 UTC from IEEE Xplore. Restrictions apply.



(a) candidate operations

(b) example of architecture

(c) padded adjacency matrix and operations

(d) vector of the architecture by flatten matrix and operations

Fig. 2. An example of our encoding of a network architecture.

Algorithm 1: Main Framework of EGFAS

Input: N: population size, T: number of iterations, N_{tra}: number of architectures in predictor's training set

Output: A^*: promising architecture

$\mathcal{S} \leftarrow$ The designed search space;
$\mathcal{A}_{\text{init}} \leftarrow$ initialization$(\mathcal{S}, N_{\text{tra}})$;
$\mathcal{A}_{\text{trained}} \leftarrow$ evaluate$(\mathcal{A}_{\text{init}})$;
$\mathcal{D}_{\text{train}} \leftarrow$ construct_training_set$(\mathcal{A}_{\text{trained}})$;
Train a predictor $\mathcal{M}$ on $\mathcal{D}_{\text{train}}$;
Initialize population $\mathcal{P}$ by randomly selecting N architectures from $\mathcal{A}_{\text{init}}$;
$t \leftarrow 0$;
while $t \leq T$ **do**
  $\mathcal{D}_{\text{test}} \leftarrow$ construct_test_set$(\mathcal{P})$;
  $Ranked_P \leftarrow$ predict_fitness$(\mathcal{M}, \mathcal{D}_{\text{test}})$;
  $\mathcal{P}_{\text{parents}} \leftarrow$ select_parents$(Ranked_P)$;
  $\mathcal{P}_{\text{offspring}} \leftarrow$ crossover_and_mutation$(\mathcal{P}_{\text{parents}})$;
  $\mathcal{P} \leftarrow$ environmental_selection$(\mathcal{P} \cup \mathcal{P}_{\text{offspring}})$;
  $t \leftarrow t + 1$;
end
$A^* \leftarrow$ Select the architecture from $\mathcal{P}$ and decode it to form the final GNN;
return A^*;

Therefore, we provide two selection operations: IDENTITY and ZERO.

- Aggregation operations: We used eleven widely adopted implementation paradigms of aggregation mechanisms as candidate aggregation operations, including gcn, gin, gat, gat_sym, gat_cos, gat_linear, gat_generalized_linear, sage, sage_sum, sage_max, and geniepath.
- Fusion operations: After selecting which predecessor nodes' outputs to use as inputs, these inputs need to be fused before aggregation. We provide eight fusion operations: max, mean, sum, concat, lstm, att, min, and hadamard (Hadamard Product).

Encoding. Fig. 2. provides an example to illustrate our encoding scheme. A candidate GNN is represented as a DAG, as shown in Fig. 2(b), which consists of an input node, an output node, and N intermediate nodes. Its topology is expressed as the lower triangular part of a $(N+2) \times (N+2)$ adjacency matrix, where the number in each cell indicates whether the row node selects the output of the column node as its input. Additionally, two $(N+2) \times 1$ type vectors are used to specify the fusion and aggregation operations for each node (with no aggregation for the output node and none for the input node), as illustrated in Fig. 2(c). Subsequently, the valid green cells are flattened in node order and converted into a one-dimensional vector of total length of $\frac{(N+2)\times(N+1)}{2} + 2N + 1$ to represent a neural architecture, as shown in Fig. 2(d).

C. Performance Predictor

In EGFAS, we designed a multi-layer perceptron (MLP) classifier as a surrogate model to predict the relative fitness between architectures. This section will introduce how the classifier is trained and makes predictions.

Training set construction. We preset to use n architectures to train the predictor. First, an initial set of architectures with size n is sampled from the proposed search space and truly trained. Unlike regression models that directly predict the absolute fitness of architectures, the classifier predicts the relative

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on May 07,2026 at 16:16:02 UTC from IEEE Xplore. Restrictions apply.

fitness between architectures by converting the comparison between two individuals into a binary classification problem. Therefore, we need to preprocess the initial set of architectures by pairing them, with each pair forming a new sample (A_1, A_2). The corresponding label can be '1' or '0', where '1' indicates that the fitness of A_1 is higher than that of A_2, and '0' indicates that A_2 has better fitness. As a result, we can obtain $\frac{(n)\times(n-1)}{2}$ samples. Finally, we use the constructed training set to train the classifier, and the loss function is defined as follows:

$$\mathcal{L} = -\frac{1}{|\mathcal{D}|} \sum_{(A_1,A_2,y)\in\mathcal{D}} \left[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}) \right], \tag{1}$$

where $\mathcal{D}$ represents the training set that we constructed, and the variables y and $\hat{y}$ correspond to true labels and predicted values of samples in $\mathcal{D}$, respectively.

Score-based fitness evaluation. In our algorithm, before each iteration begins, the trained surrogate model first performs a fitness prediction on the current population to facilitate parent selection. For the classification predictor that estimates the relative fitness of candidate architectures, we adopt a score-based evaluation method. Similar to the process of constructing the training set in the previous section, we first pair the individuals to be evaluated. For a sample (A_1, A_2), if the predicted value is 1, the score of A_1 is incremented by one; otherwise, the score of A_2 is incremented by one. By accumulating the total score of each architecture, we can easily rank them based on their total scores, which serves as the basis for parent selection.

IV. Experimental Design

A. Datasets

We evaluate the EGFAS algorithm on five graph classification benchmark datasets: Wisconsin [58], Flickr [59], Cora [60], Computers [61], Physics [62]. Where Wisconsin and Flickr are heterophilic datasets, and Cora, Computers, and Physics are homophilic. The homophily refers to the proportion of edges in a graph that connect nodes with the same class labels. And the details of the datasets are shown in Table I .

TABLE I
DETAILS OF DATASETS.

Datasets	Nodes	Edges	Features	Classes
Wisconsin	251	466	1,703	5
Flickr	89,250	899,756	500	7
Cora	2,708	5,278	1,433	7
Computers	13,381	245,778	767	10
Physics	34,493	495,924	8,415	5

B. Baseline

To validate the effectiveness of our algorithm, we set the experimental baselines into two groups: manually designed baselines and neural architecture search baselines. The manually designed group includes directly stacking GNN models (comprising four aggregation layers, denoted as Vanilla) and three commonly used predefined topological structures with fusion methods for GNNs: JK-Net, GCNII, ResGCN, we construct 4-layers GNNs on top of their framework. The NAS group includes two methods: the differentiable method SANE [21] and the RL-based method SNAG [63]. Additionally, the aggregation operations in the manually designed group are uniformly set to SAGE and GAT. The search space in the NAS group is consistent with that of EGFAS, and all focus on searching for GNNs with one input node, one output node, and four intermediate nodes.

C. Parameter Setting

For SANE and SNAG, we used the publicly available code provided by the original authors. Given the common parameter settings of DARTS [22] and RLNAS [14], as well as those specified by the original authors of SANE and SNAG, we set the search epochs of SANE to 400, after completing the search process, we select the best-performing GNN, and repeat with five different random seeds to obtain five candidates; for SNAG, we set the search epochs to 500, sampling and evaluating 10 architectures per epoch to update the controller. Finally, the controller generates five final candidates.

Given the common parameter settings of ENAS [10], for EGFAS, we set the total generations to 30, the population size to 50, the crossover rate to 0.9, and the mutation rate to 0.1 (subsequent ablation studies will explore crossover and mutation rates within [0.1, 0.3, 0.5, 0.7, 0.9]). The architecture set size for training the performance predictor is 300. After the search process, the best-performing GNN from the final population is selected, then we repeat five times to obtain five candidates.

For searched architectures by each baseline and our method, we retrain them from scratch for 400 epochs using the Adam optimizer, the weight decay $5e{-}4$ and the initial learning rate 0.01. All our experiments are performed on a single GPU card 2080Ti with pytorch.

V. Experiments Result and Analysis

A. Performance Comparisons

The experimental results are shown in Table II. We report the average test accuracy (in terms of Acc (%)) and the standard deviation of GNNs that different methods get on five datasets. In the manually designed baselines, the suffix "sage" indicates that the corresponding GNN uniformly uses sage as the aggregation function, and the suffix "gat" is the same. In addition, the best results in each group are highlighted in bold, we calculated the accuracy ranking of the eleven methods on each dataset and provided the average ranking across all datasets.

Firstly, both SANE and SNAG, as NAS-based methods, demonstrate better performance compared to many manually designed models, however, EGFAS further pushes the boundaries by leveraging evolutionary strategies to explore a broader

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on May 07,2026 at 16:16:02 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE COMPARISONS OF EGFAS AND ALL BASELINES IN TERMS OF ACC (%).

Model	Wisconsin	Flickr	Cora	Computers	Physics	Avg. Rank
Vanilla_sage	60.39(10.77)	53.07(0.50)	85.68(0.61)	90.52(0.42)	95.97(0.14)	7.4
Vanilla_gat	45.29(5.65)	50.34(2.68)	86.16(0.55)	89.08(0.43)	93.47(3.93)	9.4
JK-Net_sage	81.96(4.71)	52.41(0.33)	86.47(0.60)	91.21(0.30)	96.56(0.05)	5.0
JK-Net_gat	84.51(5.58)	53.02(0.29)	86.55(0.46)	91.80(0.23)	96.80(0.09)	3.4
GCNII_sage	81.57(4.13)	52.24(0.29)	85.83(0.42)	91.38(0.27)	96.45(0.15)	6.0
GCNII_gat	55.29(6.25)	53.03(0.29)	85.40(1.06)	91.91(0.11)	96.14(0.15)	6.6
ResGCN_sage	76.47(5.26)	53.72(0.27)	85.66(0.52)	91.51(0.18)	96.31(0.17)	5.2
ResGCN_gat	48.82(3.77)	53.63(0.24)	84.66(0.92)	90.84(0.49)	95.67(0.28)	8.2
SANE	86.47(3.09)	**53.92(0.14)**	86.40(0.38)	91.02(0.21)	96.80(0.06)	3.4
SNAG	43.92(4.65)	53.50(0.31)	84.99(1.04)	85.98(0.72)	96.18(0.11)	8.6
EGFAS(our)	**87.34(2.74)**	51.68(0.27)	**86.98(0.65)**	**92.05(0.08)**	**96.83(0.17)**	2.8

range of architectures, it achieves the best average rank among all models. EGFAS almost consistently outperforms both manually designed GNNs and other NAS-based methods across diverse datasets. These observations demonstrate the inherent advantage of NAS methods: the ability to automatically adapt and optimize the GNN architecture to suit the unique properties of each dataset. Unlike manually designed models that rely on predefined topologies and fixed aggregation functions, NAS methods explore a vast design space and can find architectures better suited for the task.

Secondly, even within the manually designed models, significant performance differences can be observed based on the choice of aggregation function. We can see that using gat as the fixed aggregation function generally outperforms sage in almost all manually designed models, but there is exceptions: in JK-Net, sage ranks higher than gat. This indicates that the aggregation function has a critical impact on the overall performance of GNNs, highlighting the importance of customizing aggregation functions for specific datasets and tasks. However, manual selection of aggregation strategies requires significant domain knowledge and may still fall short of optimal configurations, this is where NAS methods excel.

On the other hand, the datasets used in the experiments represent a mix of heterophilic and homophilic graph structures. EGFAS demonstrates strong adaptability by consistently performing well on both types of datasets, indicating its ability to handle diverse graph properties. This further underscores the limitations of manually designed models, which often fail to generalize across datasets with varying characteristics.

B. Ablation Study

In this section, we perform ablation studies for EGFAS from the evolutionary perspective. In evolutionary algorithms, the crossover rate and mutation rate are key hyperparameters. The crossover rate determines the diversity of the population, while the mutation rate greatly impacts the balance between local and global search capabilities. We define a set of values for the crossover rate and mutation rate: [0.1, 0.3, 0.5, 0.7, 0.9], and select one dataset each from homophilic and heterophilic datasets: Wisconsin and Computers, respectively, to analyze the optimal combination of these parameters under different environments.

Specifically, we perform single-variable experiments by (1) keeping the mutation rate fixed at 0.1 and gradually adjusting the crossover rate, and (2) keeping the crossover rate fixed at 0.9 and gradually adjusting the mutation rate. We then observe the impact of both on the performance of the optimal GNNs identified during the search. Each set of experiments is repeated five times, with all other hyperparameters set as described previously.

TABLE III
RESULTS OF DIFFERENT CROSSOVER RATES ON WISCONSIN AND COMPUTERS.

Crossover Rate	Wisconsin	Computers	Avg. Rank
0.9	87.34(2.74)	92.05(0.08)	3.0
0.7	**87.65(1.68)**	92.18(0.15)	1.5
0.5	86.79(2.75)	**92.31(0.07)**	3.0
0.3	87.28(1.04)	92.13(0.15)	3.0
0.1	87.12(2.13)	91.94(0.21)	4.5

TABLE IV
RESULTS OF DIFFERENT MUTATION RATES ON WISCONSIN AND COMPUTERS.

Mutation Rate	Wisconsin	Computers	Avg. Rank
0.01	85.84(2.85)	91.76(0.29)	7.0
0.05	86.24(2.44)	91.90(0.31)	5.0
0.1	**87.34(2.74)**	**92.05(0.08)**	1.0
0.3	86.88(2.32)	91.93(0.12)	3.5
0.5	87.29(2.41)	91.87(0.35)	4.5
0.7	86.69(1.95)	92.01(0.14)	3.5
0.9	87.31(1.97)	91.89(0.27)	3.5

Table III shows the results of EGFAS search with different crossover rate setups on Wisconsin and Computers datasets. It is evident that the crossover rate significantly impacts the search performance. The performance is optimal when the crossover rate is set to 0.7, achieving the best average rank across both datasets. On the other hand, lower crossover rates

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on May 07,2026 at 16:16:02 UTC from IEEE Xplore. Restrictions apply.

result in a noticeable decline in performance. This observation underscores the importance of selecting an appropriate crossover rate.

Table IV presents the results of different mutation rate setups on Wisconsin and Computers datasets. It is noteworthy that while mutation rate adjustments did not lead to drastic changes in performance, the search performance achieves its best on Wisconsin when the mutation rate is set to 0.1. However, for Computers, the difference in performance across mutation rates is relatively marginal. Interestingly, the average rank highlights that a mutation rate of 0.1 consistently performs well across both datasets, emphasizing its robustness.

However, from the existing experimental results with a set of values for the mutation rate: [0.1, 0.3, 0.5, 0.7, 0.9], it appears that the search performs better when the mutation rate is set to 0.1. This raises the question of whether a lower mutation rate is generally more beneficial for our search. However, in evolutionary algorithms, the mutation rate impacts the balance between local and global search capabilities: if it is too high, convergence becomes difficult, while if it is too low, the search may get stuck in local optima. To figure it out, we conducted five repeated experiments with mutation rates set to 0.05 and 0.01. From the results, we can see that when the mutation rate is reduced to 0.05 and 0.01, the search performance does not improve, instead, it declines. This suggests that for EGFAS, a lower mutation rate is not necessarily better.

VI. Conclusions

In this paper, an evolutionary graph fusion architecture search is proposed to over-smoothing problem automatically and efficiently. The method includes graph fusion architecture search space, a flexible encoding scheme, and a relative performance predictor. The proposed method is examined on five popular benchmark datasets and compared with many state-of-the-art algorithms and shows the effectiveness of EGFAS due to its more reasonable search space design and the fast and stable search strategy. As the experiments showing, the proposed method almost consistently outperforms other methods across diverse datasets, demonstrating a strong adaptability, which is the most important contribution of this paper.

Since the search space in this method is the basic operations, this, to some extent, might limit the structure learning ability of the graph fusion architecture. Therefore, in the future, more effective operations can be used to improve the model ability. In addition, we will leverage unsupervised learning or semi-supervised learning to reduce the need for annotation architectures for performance predictor.

Acknowledgment

This work is supported by the National Natural Science Foundation of China (No.62472079, No.62172083), the Key Technologies R&D Program of Liaoning Province (2023JH1/10400082), Guangdong Basic and Applied Basic Research Foundation (2024A1515012016), the Fundamental Research Funds for the Central Universities (N2417008).

References

[1] J. Liu, F. Xia, L. Wang, B. Xu, X. Kong, H. Tong, and I. King, "Shifu2: A network representation learning based model for advisor-advisee relationship mining," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 4, pp. 1763–1777, 2019.

[2] M. Li and Z. Zhu, "Spatial-temporal fusion graph neural networks for traffic flow forecasting," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 5, 2021, pp. 4189–4196.

[3] T. Xia and W.-S. Ku, "Geometric graph representation learning on protein structure prediction," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 1873–1883.

[4] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.

[5] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.

[6] G. Li, M. Muller, A. Thabet, and B. Ghanem, "Deepgcns: Can gcns go as deep as cnns?" in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 9267–9276.

[7] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-i. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *International conference on machine learning*. PMLR, 2018, pp. 5453–5462.

[8] M. Liu, H. Gao, and S. Ji, "Towards deeper graph neural networks," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 338–348.

[9] X. Wang and W. Zhu, "Automated machine learning on graph," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 4082–4083.

[10] Y. Sun, X. Sun, Y. Fang, G. G. Yen, and Y. Liu, "A novel training protocol for performance predictors of evolutionary neural architecture search algorithms," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 3, pp. 524–536, 2021.

[11] B. Zoph, "Neural architecture search with reinforcement learning," *arXiv preprint arXiv:1611.01578*, 2016.

[12] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "Mnasnet: Platform-aware neural architecture search for mobile," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 2820–2828.

[13] Y. Li, Z. Wen, Y. Wang, and C. Xu, "One-shot graph neural architecture search with dynamic search space," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 10, 2021, pp. 8510–8517.

[14] Y. Gao, H. Yang, P. Zhang, C. Zhou, and Y. Hu, "Graphnas: Graph neural architecture search with reinforcement learning," *arXiv preprint arXiv:1904.09981*, 2019.

[15] J. Chen, J. Gao, Y. Chen, B. M. Oloulade, T. Lyu, and Z. Li, "Autogmas: A parallel graph neural architecture search framework," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 3117–3128, 2022.

[16] S. Jiang and P. Balaprakash, "Graph neural network architecture search for molecular property prediction," in *2020 IEEE International conference on big data (big data)*. IEEE, 2020, pp. 1346–1353.

[17] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2017.

[18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.

[19] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" *arXiv preprint arXiv:1810.00826*, 2018.

[20] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 1, pp. 4–24, 2020.

[21] Z. Huan, Y. Quanming, and T. Weiwei, "Search to aggregate neighborhood for graph neural network," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 552–563.

[22] H. Liu, K. Simonyan, and Y. Yang, "Darts: Differentiable architecture search," *arXiv preprint arXiv:1806.09055*, 2018.

[23] Y. Gao, P. Zhang, C. Zhou, H. Yang, Z. Li, Y. Hu, and S. Y. Philip, "Hgnas++: efficient architecture search for heterogeneous graph neural networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 9, pp. 9448–9461, 2023.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on May 07,2026 at 16:16:02 UTC from IEEE Xplore. Restrictions apply.

[24] L. Ma, K. Hu, Y. Zhul, and H. Chen, "Computational evolution of social behavior in bacterial colony optimization model," *Journal of Pure and Applied Microbiology*, vol. 7, pp. 487–493, 2013.

[25] H. Zhang, K. Tao, L. Ma, and Y. Yong, "Handling constrained multi-objective optimization with objective space mapping to decision space based on extreme learning machine," in *2020 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2020, pp. 1–7.

[26] X. Zhou, A. K. Qin, M. Gong, and K. C. Tan, "A survey on evolutionary construction of deep neural networks," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 5, pp. 894–912, 2021.

[27] M. Nunes, P. M. Fraga, and G. L. Pappa, "Fitness landscape analysis of graph neural network architecture search spaces," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2021, pp. 876–884.

[28] N. Li, L. Ma, G. Yu, B. Xue, M. Zhang, and Y. Jin, "Survey on evolutionary deep learning: Principles, algorithms, applications, and open issues," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–34, 2023.

[29] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *International conference on machine learning*. PMLR, 2018, pp. 4095–4104.

[30] Z. Lu, I. Whalen, Y. Dhebar, K. Deb, E. D. Goodman, W. Banzhaf, and V. N. Boddeti, "Multiobjective evolutionary design of deep convolutional neural networks for image classification," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 2, pp. 277–291, 2020.

[31] Y. Chen, G. Meng, Q. Zhang, S. Xiang, C. Huang, L. Mu, and X. Wang, "Renas: Reinforced evolutionary neural architecture search," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 4787–4796.

[32] Y. Liu, Y. Sun, B. Xue, M. Zhang, G. G. Yen, and K. C. Tan, "A survey on evolutionary neural architecture search," *IEEE transactions on neural networks and learning systems*, vol. 34, no. 2, pp. 550–570, 2021.

[33] K. Zhou, X. Huang, Q. Song, R. Chen, and X. Hu, "Auto-gnn: Neural architecture search of graph neural networks," *Frontiers in big Data*, vol. 5, p. 1029307, 2022.

[34] Z. Lu, R. Cheng, Y. Jin, K. C. Tan, and K. Deb, "Neural architecture search as multiobjective optimization benchmarks: Problem formulation and performance assessment," *IEEE transactions on evolutionary computation*, vol. 28, no. 2, pp. 323–337, 2023.

[35] Y. Yao, X. Wang, Y. Qin, Z. Zhang, W. Zhu, and H. Mei, "Data-augmented curriculum graph neural architecture search under distribution shifts," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 15, 2024, pp. 16 433–16 441.

[36] Y. Gao, H. Yang, Y. Chen, J. Wu, P. Zhang, and H. Wang, "Llm4gnas: A large language model based toolkit for graph neural architecture search," *arXiv preprint arXiv:2502.10459*, 2025.

[37] Q. Wang, Y. Shao, Y. Ma, and M. Liu, "Nodenas: Node-specific graph neural architecture search for out-of-distribution generalization," *arXiv preprint arXiv:2503.02448*, 2025.

[38] G. Corso, L. Cavalleri, D. Beaini, P. Liò, and P. Veličković, "Principal neighbourhood aggregation for graph nets," *Advances in Neural Information Processing Systems*, vol. 33, pp. 13 260–13 271, 2020.

[39] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, H. Harutyunyan, G. Ver Steeg, and A. Galstyan, "Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing," in *international conference on machine learning*. PMLR, 2019, pp. 21–29.

[40] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, "Simple and deep graph convolutional networks," in *International conference on machine learning*. PMLR, 2020, pp. 1725–1735.

[41] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter, "Nas-bench-101: Towards reproducible neural architecture search," in *International conference on machine learning*. PMLR, 2019, pp. 7105–7114.

[42] W. Wen, H. Liu, Y. Chen, H. Li, G. Bender, and P.-J. Kindermans, "Neural predictor for neural architecture search," in *European Conference on Computer Vision*. Springer, 2020, pp. 660–676.

[43] X. Ning, Y. Zheng, T. Zhao, Y. Wang, and H. Yang, "A generic graph-based neural architecture encoding scheme for predictor-based nas," in *European Conference on Computer Vision*. Springer, 2020, pp. 189–204.

[44] M. Chatzianastasis, G. Dasoulas, G. Siolas, and M. Vazirgiannis, "Graph-based neural architecture search with operation embeddings," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 393–402.

[45] E.-G. Talbi, "Optimization of deep neural networks: a survey and unified taxonomy," 2020.

[46] C. Wei, C. Niu, Y. Tang, Y. Wang, H. Hu, and J. Liang, "Npenas: Neural predictor guided evolution for neural architecture search," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 11, pp. 8441–8455, 2022.

[47] C. White, W. Neiswanger, and Y. Savani, "Bananas: Bayesian optimization with neural architectures for neural architecture search," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 12, 2021, pp. 10 293–10 301.

[48] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[49] E. Real, S. Moore, A. Selle, S. Saxena, Y. L. Suematsu, J. Tan, Q. V. Le, and A. Kurakin, "Large-scale evolution of image classifiers," in *International conference on machine learning*. PMLR, 2017, pp. 2902–2911.

[50] H. Liu, K. Simonyan, O. Vinyals, C. Fernando, and K. Kavukcuoglu, "Hierarchical representations for efficient architecture search," *arXiv preprint arXiv:1711.00436*, 2017.

[51] Y. Peng, A. Song, V. Ciesielski, H. M. Fayek, and X. Chang, "Prenas: Evolutionary neural architecture search with predictor," *IEEE Transactions on Evolutionary Computation*, vol. 27, no. 1, pp. 26–36, 2022.

[52] Y. Xue and A. Zhu, "An effective surrogate-assisted rank method for evolutionary neural architecture search," *Applied Soft Computing*, vol. 167, p. 112392, 2024.

[53] L. Ma, N. Li, P. Zhu, K. Tang, A. Khan, F. Wang, and G. Yu, "A novel fuzzy neural network architecture search framework for defect recognition with uncertainties," *IEEE Transactions on Fuzzy Systems*, 2024.

[54] Y. Sun, H. Wang, B. Xue, Y. Jin, G. G. Yen, and M. Zhang, "Surrogate-assisted evolutionary deep learning using an end-to-end random forest-based performance predictor," *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 2, pp. 350–364, 2019.

[55] Y. Xu, Y. Wang, K. Han, Y. Tang, S. Jui, C. Xu, and C. Xu, "Renas: Relativistic evaluation of neural architecture search," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 4411–4420.

[56] L. Ma, N. Li, G. Yu, X. Geng, S. Cheng, X. Wang, M. Huang, and Y. Jin, "Pareto-wise ranking classifier for multi-objective evolutionary neural architecture search," *IEEE Transactions on Evolutionary Computation*, 2023.

[57] L. Ma, H. Kang, G. Yu, Q. Li, and Q. He, "Single-domain generalized predictor for neural architecture search system," *IEEE Transactions on Computers*, 2024.

[58] H. Pei, B. Wei, K. C.-C. Chang, Y. Lei, and B. Yang, "Geom-gcn: Geometric graph convolutional networks," *arXiv preprint arXiv:2002.05287*, 2020.

[59] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna, "Graphsaint: Graph sampling based inductive learning method," *arXiv preprint arXiv:1907.04931*, 2019.

[60] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI magazine*, vol. 29, no. 3, pp. 93–93, 2008.

[61] J. McAuley, C. Targett, Q. Shi, and A. Van Den Hengel, "Image-based recommendations on styles and substitutes," in *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*, 2015, pp. 43–52.

[62] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," *arXiv preprint arXiv:1811.05868*, 2018.

[63] H. Zhao, L. Wei, and Q. Yao, "Simplifying architecture search for graph neural network," *arXiv preprint arXiv:2008.11652*, 2020.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on May 07,2026 at 16:16:02 UTC from IEEE Xplore. Restrictions apply.